安全方案

1. 文档概述

1.1 文档目的

本文档用于定义ngu800P的安全方案，目标是在现有 SoC 架构基础上，结合新增的 eHSM 安全子系统 IP，形成一套可工程落地、可评审、可用于流片前需求收集的整体方案，并覆盖以下内容：

1. 系统级信任根与安全边界
2. 安全启动与非安全启动路径
3. SoC BootROM、SEC1、SEC2、后续微核以及 eHSM 的职责划分
4. 固件镜像格式、验签链、防回滚机制
5. Host 通过 PCIe 下发后续固件的安全控制机制
6. 安全升级、设备认证、调试鉴权与生命周期管理
7. 流片前必须冻结的 RTL 要求
8. 客户安全需求与设计实现的映射关系

1.2 适用范围

本文档适用于以下模块：

- SoC BootROM
- SEC 核
- SEC1 固件
- SEC2 固件
- PM 微核
- RAS 微核
- Codec 微核 1
- Codec 微核 2
- eHSM 安全子系统
- Host 驱动 / Host 安全代理

1.3 设计前提

本方案基于以下明确约束：

1. **SoC BootROM 无能力、无权限直接对 SEC1 做密码学验签**
2. **SEC1 固件位于 Flash 中**
3. **SEC1 负责最基础平台初始化，如 PCIe 初始化**
4. **SEC2 及后续微核固件由 Host 通过 PCIe 下发**
5. **SPDM / PLDM 软件栈运行在 SEC2 侧**
6. **SEC 侧涉及签名验签、HASH、加解密、随机数、密钥、OTP、生命周期、调试 challenge/鉴权等能力时，通过 mailbox 调用 eHSM**
7. **非安全启动路径应允许系统不依赖 eHSM 正常启动**
8. **安全启动路径中，SEC1 的首次密码学校验由 eHSM 完成，而不是由 BootROM 直接完成**

2. 总体架构原则

2.1 总体设计原则

本方案采用如下总体结构：

SoC BootROM 负责启动编排 + SEC 负责安全控制面 + eHSM 提供安全服务后端

其核心思想为：

- **SoC BootROM** 是最早执行代码，但不承担镜像密码学校验能力
- **eHSM** 是系统首次可用的密码学验证根
- **SEC1** 是完成基础 bring-up 的最小初始化固件
- **SEC2** 是完整安全控制固件，负责后续固件接收、验证、升级、认证与调试控制
- **Host** 通过 PCIe 下发 SEC2 及其他微核固件，但只拥有“投递能力”，不拥有“执行放行权”

2.2 安全控制面与安全服务面的分离

本方案明确将系统安全逻辑分为两层：

A. 安全控制面：SEC

由 SEC1/SEC2 承担，主要负责：

- 启动链控制
- PCIe 通道建立
- Host 固件接收
- 微核固件验证编排
- 升级管理

- 调试鉴权控制
- SPDM / PLDM 协议处理
- measurement_table 汇总与证明报告组装

B. 安全服务面：eHSM

由 eHSM 提供，主要负责：

- 安全启动相关镜像验证能力
- 密钥管理
- 签名验签
- HASH / HMAC / MAC
- 对称加解密 / AEAD
- TRNG 随机数
- OTP / 生命周期访问
- 调试 challenge / 调试鉴权
- 计数器 / 时间等可信基础服务

eHSM 文档明确说明，其作为片上信任根子系统，具备 32 位 RISC-V CPU、对称/公钥/哈希引擎、TRNG、安全控制、生命周期、安全启动、密钥管理、身份认证、固件升级、调试保护等能力，并通过 Mailbox与 SoC 交互。

Host API 手册也明确列出了随机数、HASH、对称加解密、SM2 签名验签、密钥管理、镜像验证、升级、计数器、OTP 读写、challenge、debug_auth、lifecycle 修改等能力。

3. Root of Trust 与安全边界定义

3.1 系统级最早执行起点

系统上电后，最早执行的代码是：

SoC BootROM / eHSM bootloader(rom)

3.2 系统级首次密码学验证根

由于 SoC BootROM 无能力、无权限直接完成 SEC1 的密码学校验，因此系统第一次真正对“下一阶段代码是否可信”做签名/版本/吊销判断的主体是：

eHSM

因此，在安全启动路径下：

- **BootROM 是启动编排起点**

- **eHSM 是首个可用的密码学验证根**

3.3 eHSM 内部可信起点

eHSM 本身具有 IROM、Bootloader、FW 等内部启动链，其自身启动流程包括硬件 Boot、Bootloader 执行、eHSM FW 执行，用于完成自检、密钥装载、生命周期恢复、固件校验等。

因此，eHSM 内部也有其自身 Root of Trust，但该 Root 的作用范围是：

- eHSM 子系统内部可信启动
- eHSM 安全服务能力可用性

而不是整颗 SoC 上电后的最早执行入口。

3.4 本方案中的正式口径

在安全启动模式下，

- SoC BootROM 作为系统最早执行代码，负责启动编排；
- eHSM 作为首个可用的密码学验证根，负责对 SEC1 镜像执行签名校验、版本检查、吊销检查及可选解密；
- BootROM 仅在收到 eHSM 验证通过结果后，才装载并启动 SEC1。

4. 模块职责划分

4.1 SoC BootROM

SoC BootROM 职责限定为：

1. 上电最小初始化
2. 启动模式分流（安全 / 非安全）
3. 拉起 eHSM 启动链并等待其进入可服务状态
4. 从 Flash 定位 SEC1 镜像
5. 向 eHSM 发起 SEC1 验证请求
6. 根据 eHSM 返回结果决定是否装载并启动 SEC1
7. 记录早期启动状态与错误码
8. 在安全路径中，不直接承担密码学验签与解密逻辑

4.2 SEC1

SEC1 定义为：

位于 Flash 中的最小初始化固件

其职责包括：

1. 最基础平台初始化
2. PCIe 初始化
3. Host 通道建立
4. 共享缓冲区 / doorbell / mailbox 等基础通信环境建立
5. 为 SEC2 与其他后续固件下载准备运行环境

SEC1 不承担：

- 完整 SPDM / PLDM 协议栈
- 完整认证报告生成
- 完整升级状态机
- 所有微核的长期安全策略管理

4.3 SEC2

SEC2 定义为：

完整安全控制固件

其职责包括：

1. 安全控制平面初始化
2. 初始化 SEC ↔ eHSM mailbox 通道
3. 启动 SPDM responder
4. 可选启动 PLDM 固件更新管理
5. 接收 Host 下发的后续微核固件
6. 调用 eHSM 完成镜像验签 / HASH / 可选解密
7. 维护 measurement_table
8. 组装并签名 attestation report
9. 控制升级、回滚、恢复
10. 控制调试鉴权
11. 管理生命周期相关策略执行

4.4 eHSM BootLoader (提供启动阶段能力)

可以理解为：**启动阶段的最小可信执行环境**

1. 自检（Self-Test）
    - 算法自检（SM2/SM3/SM4/TRNG 等）

2. 镜像验证（Boot Stage）
3. 固件认证（Boot Authentication）
    - challenge
    - firmware auth（国密）
    - debug auth
4. 密钥初始化能力
    - 随机密钥生成（用于 OTP）
    - 外部 key 加密
5. OTP / 寄存器操作
6. 基础系统控制
    - UART
    - 基础配置

4.5 eHSM fw

1. 密码学基础服务（Crypto Service）
    - 随机数
    - Hash / HMAC / MAC
    - 对称加密
    - 非对称加密 + 签名
2. 密钥管理（Key Management）
    - 密钥生成
    - 导入 / 导出
    - 删除
    - 派生（KDF）
    - 密钥交换（DH / SM2 / SM9）
3. 镜像管理（Runtime Image Service）
    - 镜像验证
    - 固件升级（完整三段式）
    - anti-rollback（结合 counter）
4. 设备认证 & SPDM 支撑
    - challenge 生成

    - 签名能力
    - key 使用
5. 生命周期与系统控制
    - lifecycle 切换（DEV / PROD / RMA）
    - control field 修改
    - debug 鉴权
    - debug 开关
6. OTP / Counter / Time / 状态
    - OTP 读写
    - counter（rollback）
    - UTC 时间
    - version

4.6 后续微核

PM / RAS / Codec 微核职责包括：

- 接收由 SEC2 校验通过的固件
- 在未被 SEC2 放行前不得执行
- 不直接拥有根密钥与信任裁决权
- 必要时向 SEC2 上报运行状态和异常信息

5. 启动路径设计

5.1 非安全启动路径

5.1.1 目标

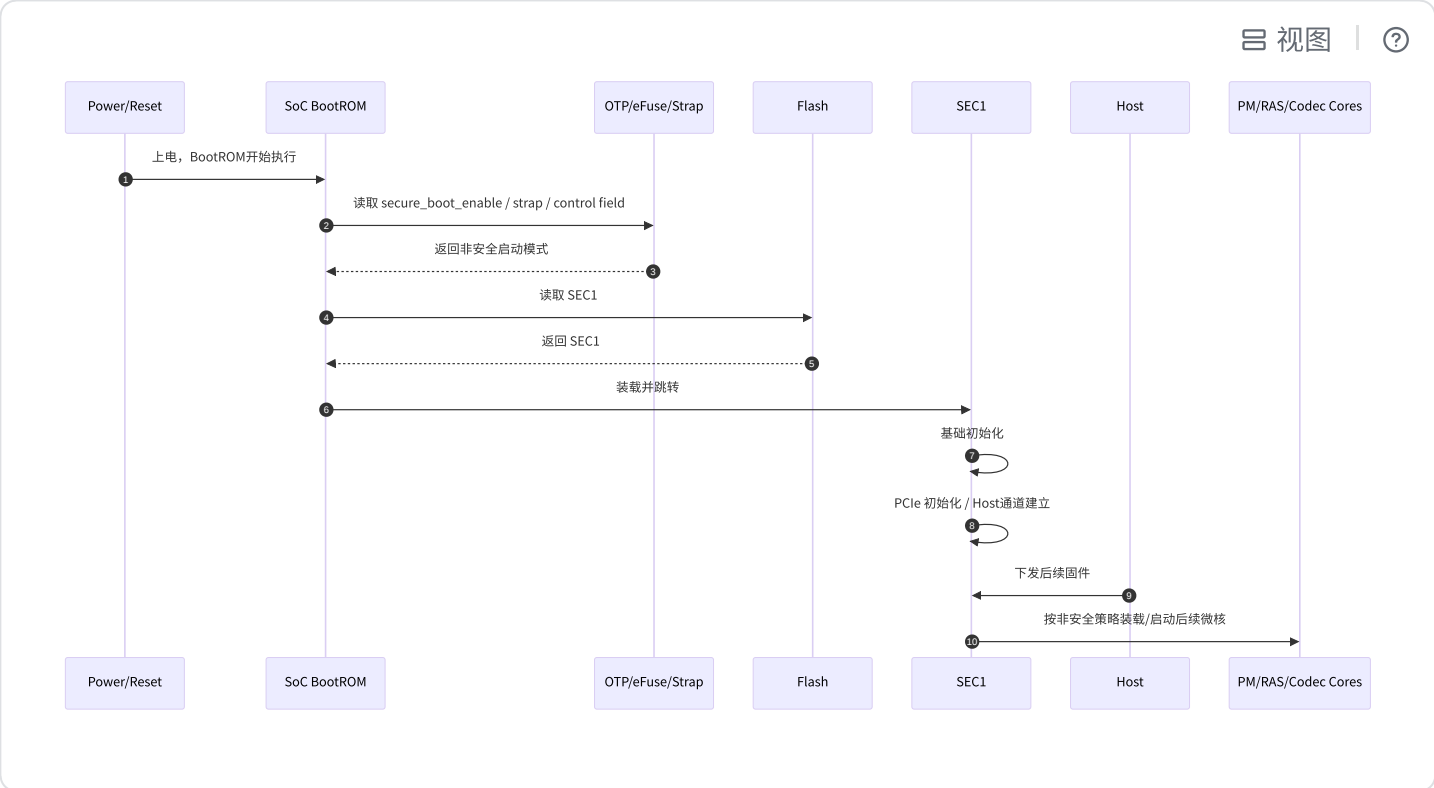
用于：

- bring-up
- 开发调试模式
- 客户不启用安全能力的模式

5.1.2 流程

1. SoC BootROM 上电执行
2. 判定当前为非安全启动模式

3. 直接装载 Flash 中的 SEC1
4. SEC1 完成 PCIe 初始化与基础 bring-up
5. 后续固件按非安全策略加载
6. 系统进入普通运行态



5.1.3 特点

- 不强制依赖 eHSM
- 不提供可信链保证
- 不满足客户对安全启动、可信升级、部件度量认证的正式要求

5.2 安全启动路径

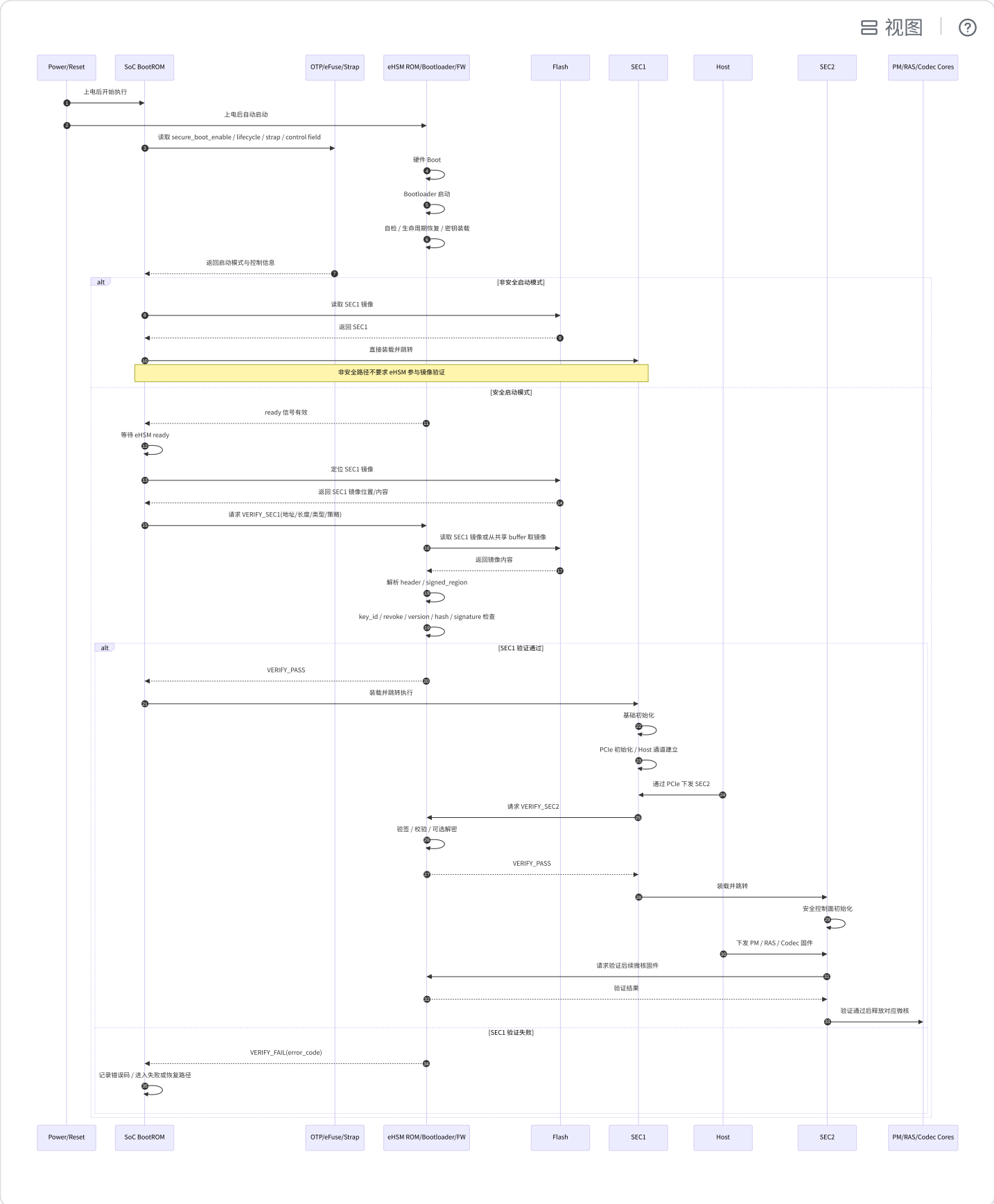
5.2.1 目标

在安全启动模式下，满足：

1. SEC1 必须在执行前经 eHSM 验证
2. 后续固件必须经 SEC2 调用 eHSM 验证
3. Host 下发固件在执行前必须受控
4. 支持版本检查、防回滚、吊销
5. 支持设备认证与度量导出

6. 支持安全升级与安全调试

5.2.2 启动流程



阶段 A：SoC BootROM 启动

1. 上电后 SoC BootROM 运行

2. 完成最小平台初始化
3. 读取 secure boot enable / lifecycle / control field / strap
4. 判定进入安全启动模式

阶段 B（与A同步）：eHSM 启动

5. SoC BootROM 拉起 eHSM
6. eHSM 完成自身启动链：
    - 硬件 Boot
    - Bootloader 执行
    - 自检
    - OTP/eFuse 配置装载
    - 生命周期恢复
    - 密钥材料恢复

阶段 C：SEC1 验证

7. SoC BootROM 从 Flash 定位 SEC1 镜像
8. SoC BootROM 通过 mailbox 向 eHSM 发起 VERIFY_SEC1 请求
9. 请求包含：
    - 镜像地址或缓冲区地址
    - 镜像长度
    - 镜像类型
    - 可选 load address
    - 可选版本检查参数
10. eHSM 对 SEC1 执行：

- header 解析
- signer key slot / key_id 检查
- revoke bitmap 检查
- version / min rollback 检查
- signer pubkey hash 校验
- signature 校验
- payload hash 校验
- 可选解密

11. eHSM 返回验证结果

阶段 D：SEC1 装载与启动

12. 若 eHSM 返回通过：

- SoC BootROM 装载 SEC1 到目标执行区
- 或使用 eHSM 输出的解包/解密结果

13. SoC BootROM 跳转执行 SEC1

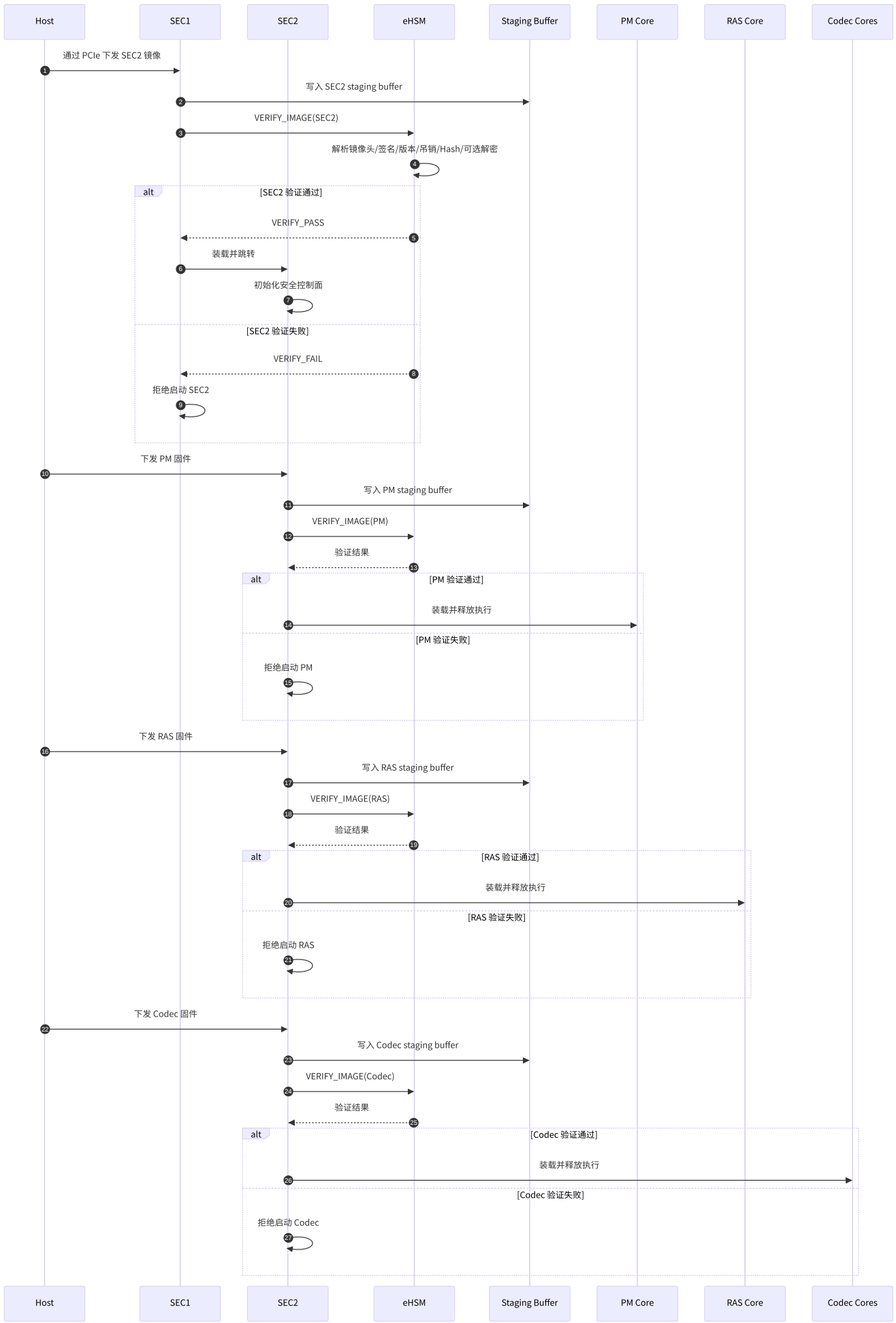
阶段 E：SEC1 基础初始化

14. SEC1 完成最基础初始化
15. 完成 PCIe 初始化
16. 建立 Host 通道和共享缓冲区

阶段 F：SEC2 及后续固件加载

17. Host 通过 PCIe 下发 SEC2、PM、RAS、Codec 等固件
18. SEC1/SEC2 调用 eHSM 对各镜像进行验签、HASH 与可选解密
19. 验证通过后，SEC2 更新 measurement_table
20. SEC2 释放对应微核执行

视图


Host
SEC1
SEC2
eHSM
Staging Buffer
PM Core
RAS Core
Codec Cores
1 通过 PCIe 下发 SEC2 镜像
2 写入 SEC2 staging buffer
3 VERIFY_IMAGE(SEC2)
4 解析镜像头/签名/版本/吊销/Hash/可选解密
alt
[SEC2 验证通过]
5 VERIFY_PASS
6 装载并跳转
7 初始化安全控制面
[SEC2 验证失败]
8 VERIFY_FAIL
9 拒绝启动 SEC2
10 下发 PM 固件
11 写入 PM staging buffer
12 VERIFY_IMAGE(PM)
13 验证结果
alt
[PM 验证通过]
14 装载并释放执行
[PM 验证失败]
15 拒绝启动 PM
16 下发 RAS 固件
17 写入 RAS staging buffer
18 VERIFY_IMAGE(RAS)
19 验证结果
alt
[RAS 验证通过]
20 装载并释放执行
[RAS 验证失败]
21 拒绝启动 RAS
22 下发 Codec 固件
23 写入 Codec staging buffer
24 VERIFY_IMAGE(Codec)
25 验证结果
alt
[Codec 验证通过]
26 装载并释放执行
[Codec 验证失败]
27 拒绝启动 Codec
Host
SEC1
SEC2
eHSM
Staging Buffer
PM Core
RAS Core
Codec Cores

阶段 G：运行态

21. SEC2 常驻
22. 对外提供：

- SPDM responder
- 安全升级
- 调试鉴权
- 状态查询
- attestation report 输出

6. 固件分层与镜像定义（以国密体系为例）

eHSM有推荐的固件格式，包括加密算法，证书关键字段等，但**目前未拿到具体信息**；先按照满足字节需求以及前期竞品调研得结果来进行固件格式设计，后续可以跟eHSM的要求进行调整。

6.1 镜像分层

本方案将固件分为以下几类：

1. **SEC1**：Flash 中的基础初始化固件
2. **SEC2**：完整安全控制固件
3. **后续微核固件**：PM / RAS / Codec
4. **eHSM FW**：eHSM常驻的提供安全服务的固件
5. **敏感 blob**：策略表、参数表、特殊微码等

6.2 通用镜像格式

结合ngu800的实际需求及对竞品的调研分析，固件的加密格式可主要包括 **header + Signed Region + signature + wrapped_cek + 加密原始镜**五个部分。平台侧只对每个发布包生成随机 CEK，设备侧只保存长期 FW_KEK slot，从而兼顾批量分发与密钥隔离。

6.2.1 header

可参考edge10的头格式及加密所需的信息，header可包含以下内容

代码块

```
struct common_firmware_header {
    uint32_t magic;              // 固定魔数
    uint32_t header_size_bytes;  // header大小
```

```
    uint32_t total_size_bytes;    // 整包大小

    uint32_t signed_region_offset;
    uint32_t signed_region_size;      // 被签名区域长度
    uint32_t signature_offset;
    uint32_t wrapped_cek_offset;     // 包裹后的 CEK 位置
    uint32_t wrapped_cek_size;         // 包裹后的 CEK 长度
    uint32_t enc_payload_offset;     // 密文 payload 位置
    uint32_t enc_payload_size;      // 密文 payload 大小

    uint32_t flags;               // 压缩/是否加密/调试标志等
    uint8_t  iv[16];              // 先按16字节放，便于SM4-CTR/CBC
};
```

- **header_size_bytes**
  - header 自己的大小，便于软件读取header信息
- **iv**
  - 初始向量
  - 对称加解密使用
- **size_signed**
  - 被签名覆盖的长度
- **wrapped_cek_offset / wrapped_cek_size**
  - 明确包裹后的 CEK 在固件包中的位置，便于BootROM 在完成验签后快速定位并执行unwrap。
- **enc_payload_offset / enc_payload_size**
  - 指向由 CEK 加密后的安全核正文。

6.2.2 Signed Region && signature

Signed Region 是签名覆盖的主区域，用于绑定版本、最小允许版本、key_id、payload_hash 以及与

本包相关的密钥元数据。

代码块

```
#define FW_NAME_MAX 16
#define PUBKEY_MAX  128
#define HASH_LEN    32    // SM3

struct signed_region_v1 {
```

```
    uint32_t sr_version;          // signed region版本
    uint32_t firmware_type;       // SEC/PMU/...
    uint32_t ucode_version;       // 当前固件版本
    uint32_t min_rollback_ver;    // 最低允许版本

    uint32_t load_addr;
    uint32_t entry_point;
    uint32_t payload_size;
    uint32_t algo_type;           // SM2+SM3+SM4

    uint32_t key_id;              // 用于后续密钥轮换
    uint32_t valid_from;          // 可选
    uint32_t valid_to;            // 可选
    uint32_t flags;               // 压缩/加密模式等

    uint16_t pubkey_len;
    uint16_t payload_hash_len;
    uint16_t wrapped_cek_hash_len;
    uint16_t reserved0;
    uint8_t  signer_pubkey[PUBKEY_MAX]; // SM2公钥
    uint8_t  payload_hash[HASH_LEN];    // 明文payload的SM3
    uint8_t  wrapped_cek_hash[HASH_LEN];
};
```

- 被签名区域主要的内容包括
  - 固件版本号
  - 固件类型（PMU/SEC/CODEC）
  - 加载策略
  - 有效日期
  - 区域大小信息
  - 可能的 rollback 防护版本
  - 算法类型
  - **公钥**
  - **payload的hash值**
  - wrapped_cek_hash：防止攻击者在不改变 payload 的情况下替换包中的 wrapped_cek。
- 签名
  - **对上述Signed Region的签名值**
  - 用于证明上述内容的合法性与完整性

6.2.3 wrapped_cek 与 enc_payload

- 平台侧每次打正式包时，随机生成一把 CEK（content encryption key）。
- 使用 CEK + IV 对安全核明文 payload 加密，生成 enc_payload。
- 再使用与 key_id 对应的 FW_KEK slot 对 CEK 执行包裹，生成 wrapped_cek。
- 最终固件包中不包含 CEK 明文，只携带 wrapped_cek。设备侧使用安全存储中的 FW_KEK slot 恢复

6.3 其他固件

6.3.1 加密格式

除安全核固件外，其他固件不涉及安全策略及根密钥信息，同时考虑到固件大小可能会偏大，加密格式可只做签名，不做主体镜像的明文加密。其他头信息按上述格式统一。

6.3.2 可选的敏感 blob 加密

对少量特别敏感的微码、表项或训练参数 blob，可按与安全核一致的方式扩展为：`header + Signed Region + signature + wrapped_cek + enc_payload`，并由 freertos(sec) 在运行阶段按 key_id 执行 unwrap 与解密

6.4 efuse中需包含的密钥信息

eHSM中有对efuse内容和布局的要求，其功能可覆盖我司方案的基本需求，这里只针对我司方案中需要用到的密钥

对象	是否设备唯一	主要用途	是否可导出
UDS	是	设备身份根、本地 sealing	否
FW_KEK Slot0..N	否（按产品/ 批次）	unwrap wrapped_cek，保护固件内容密钥	否
Signer Pubkey Hash Slot0..N	否	校验 Signed Region 中 signer_pubkey 合法性	否
Revoke Bitmap/Active Key Policy	否	密钥撤销、轮换与量产态策略控制	否
Debug Root / CA Hash	否	安全 Debug 凭证验证	否

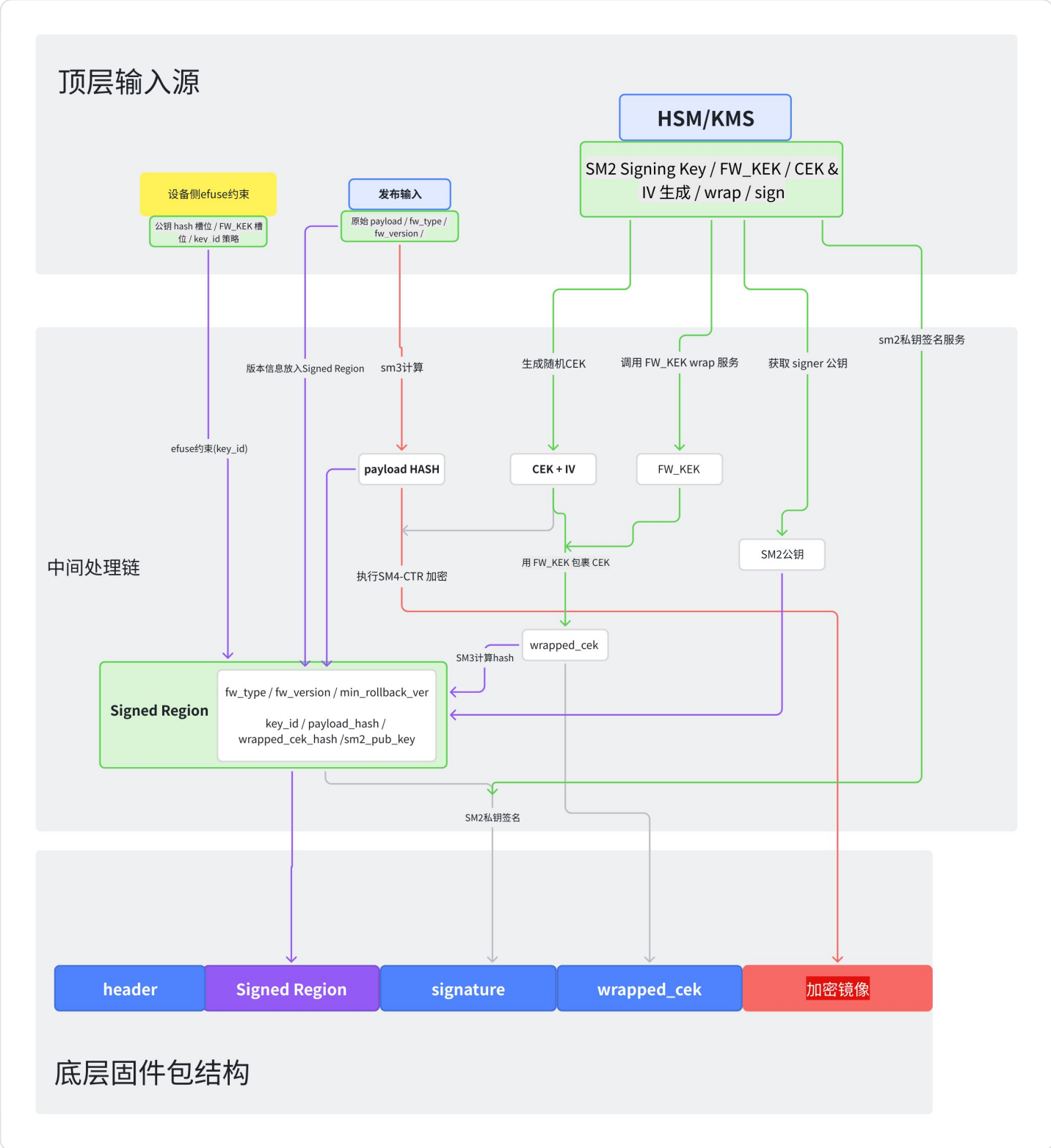
7. 镜像验证

7.1 加解密流程

7.1.1 平台侧固件制作流程

安全核固件制作流程如下：

1. 生成安全核明文 payload，并计算 payload_hash = SM3(payload)。
2. 生成随机 CEK（每包一把），并生成 IV。
3. 用 CEK + IV 对 payload 执行 SM4-CTR加密，得到 enc_payload。
4. 按待发布的产品线、SKU 或批次选择 key_id；平台侧使用与该 key_id 对应的 FW_KEK 或其等效封装服务，对 CEK 执行 Wrap，得到 wrapped_cek。
5. 计算 wrapped_cek_hash = SM3(wrapped_cek)，并将 key_id、payload_hash、wrapped_cek_hash、版本号、最小允许版本等写入 Signed Region。
6. 将 signer_pubkey 放入 Signed Region，对 Signed Region 执行 SM2 签名，得到 signature。
7. 输出最终固件包：header + Signed Region + signature + wrapped_cek + enc_payload。
8.

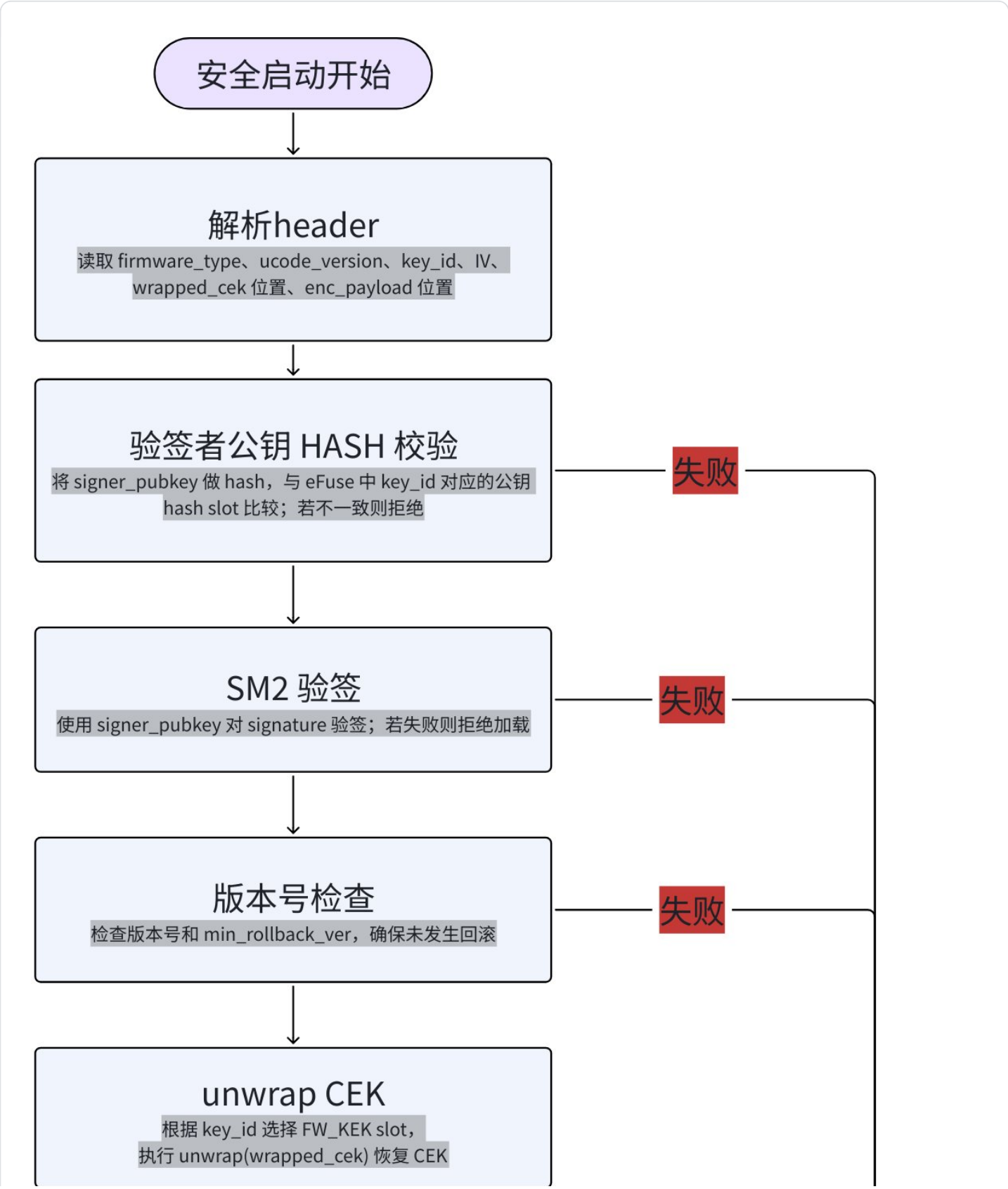


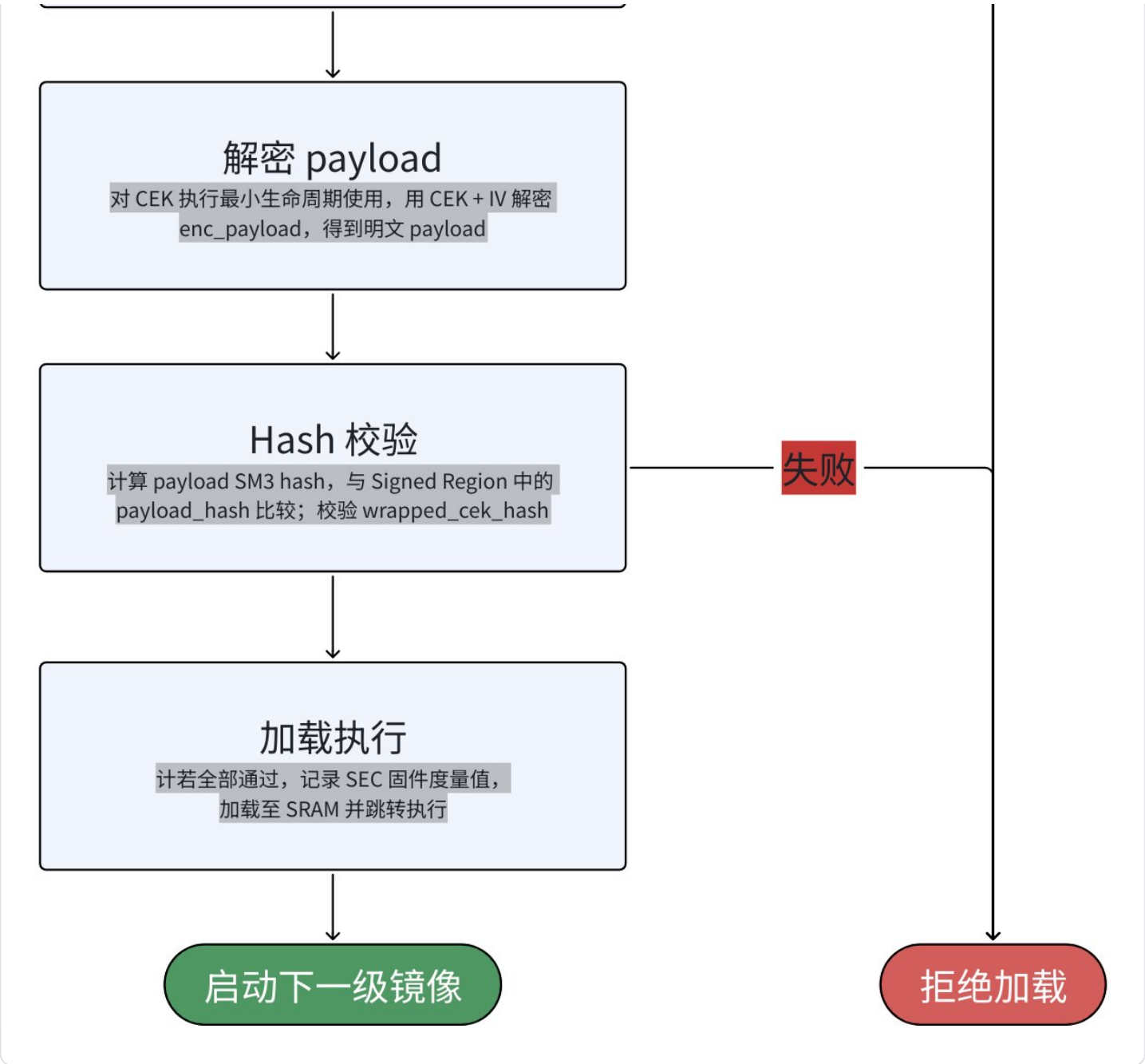
7.1.2 设备侧验证与解包流程

设备上电后，BootROM 从flash中读取sec1，提交验签和解密申请，eHSM对固件进行解密验签：

1. 解析 header，读取 firmware_type、ucode_version、key_id、IV、wrapped_cek 位置、enc_payload 位置等信息。
2. 将 Signed Region 中的 signer_pubkey 做 hash 计算，并与 eFuse 中与 key_id 对应的公钥 hash slot 比较；若不一致，则拒绝加载。
3. 使用 signer_pubkey 对 signature 验签；若验签失败，则拒绝加载。

4. 检查版本号和 min_rollback_ver，确保未发生回滚。
5. 根据 key_id 在设备侧选择 FW_KEK slot，执行 unwrap(wrapped_cek) 恢复 CEK。
6. 对恢复出的 CEK 执行最小生命周期使用，并用 CEK + IV 解密 enc_payload，得到明文 payload。
7. 计算 payload 的 SM3 hash，并与 Signed Region 中的 payload_hash 比较；对 wrapped_cek 计算 SM3，得到 `wrapped_cek_hash`，并与 `Signed Region` 中的对应字段比较。
8. 若全部通过，则记录 SEC 固件度量值，加载至 SRAM 并跳转执行。





7.2 根密钥注入与生命周期

“设备唯一根”与“固件分发根”在注入方式上不同：

- HUK：由片内 TRNG 在安全 provisioning 状态下生成，并写入 eFuse。外部系统只接收其公钥、CSR 或摘要，不获取明文。
- FW_KEK slot：按产品/客户批次由受控 provisioning 流程注入设备安全区，可支持 2~4 个 slot，配套 revoke bit 与 key_id，以便量产后轮换和应急切换。
- 进入 PROD 生命周期后，禁止重新注入 HUK；FW_KEK 的更新仅允许在受控 RMA / 工厂流程中按策略执行。

7.3 SEC2 与后续固件验证

SEC2 及 PM / RAS / Codec 固件由 SEC2 调用 eHSM 完成验证。

SEC2 是安全控制主体，eHSM 是底层安全能力提供者。

8. eHSM 安全服务接口使用原则

8.1 调用方式

SEC1/SEC2 与 eHSM 之间统一采用：

- Mailbox 命令
- 共享缓冲区 / 共享内存

这与 eHSM 文档中的交互模型一致：SoC CPU 通过 Mailbox 与 eHSM 进行命令交互，通过共享 memory 进行数据传输。

8.2 建议使用的能力

SEC 侧优先调用如下能力：

- 随机数：`ehsm_gen_random`
- HASH：`ehsm_hash_*`
- 对称加解密：`ehsm_symm_cipher_*`
- AEAD：`ehsm_aead_*`
- SM2 签名验签：`ehsm_sm2_sign_*`
- 密钥管理：`ehsm_km_*`
- 镜像验证：`ehsm_verify_image`
- 升级：`ehsm_upgrade_fw_image_*`
- 计数器：`ehsm_create_counter` / `read_counter` / `increase_counter`
- OTP / 生命周期：`ehsm_read_otp` / `ehsm_write_otp` / `ehsm_change_lifecycle`
- 调试鉴权：`ehsm_get_challenge` / `ehsm_debug_auth` / `ehsm_close_debug`

9. Host 通过 PCIe 下发固件的安全控制

9.1 基本原则

在本架构下：

- Host 可以通过 PCIe 将固件写入 buffer
- 但 Host 不拥有任何微核的执行放行权

- 是否允许执行，由 SEC2 决定
- 验证能力由 SEC2 调用 eHSM 完成

9.2 强制区分三个动作

必须严格区分以下三个阶段：

1. **镜像投递**
2. **镜像验证**
3. **执行放行**

不能因为 Host 已将固件写入某个缓冲区，就自动视为固件可信。

9.3 RTL 门控要求

必须提供以下控制信号，并确保仅 SEC 可写：

- PM `reset_release`
- RAS `reset_release`
- Codec1 `reset_release`
- Codec2 `reset_release`
- `fetch_enable`
- `boot_go`
- 可选 `boot address lock`

默认状态应为：

- 不释放
- 不允许取指执行

只有 SEC2 在验证通过后，才能放行。

10. 安全升级方案

10.1 目标

满足客户关于升级可信根、禁止绕过升级路径、防止不可恢复 DoS 和 bootkit 的要求。

10.2 升级原则

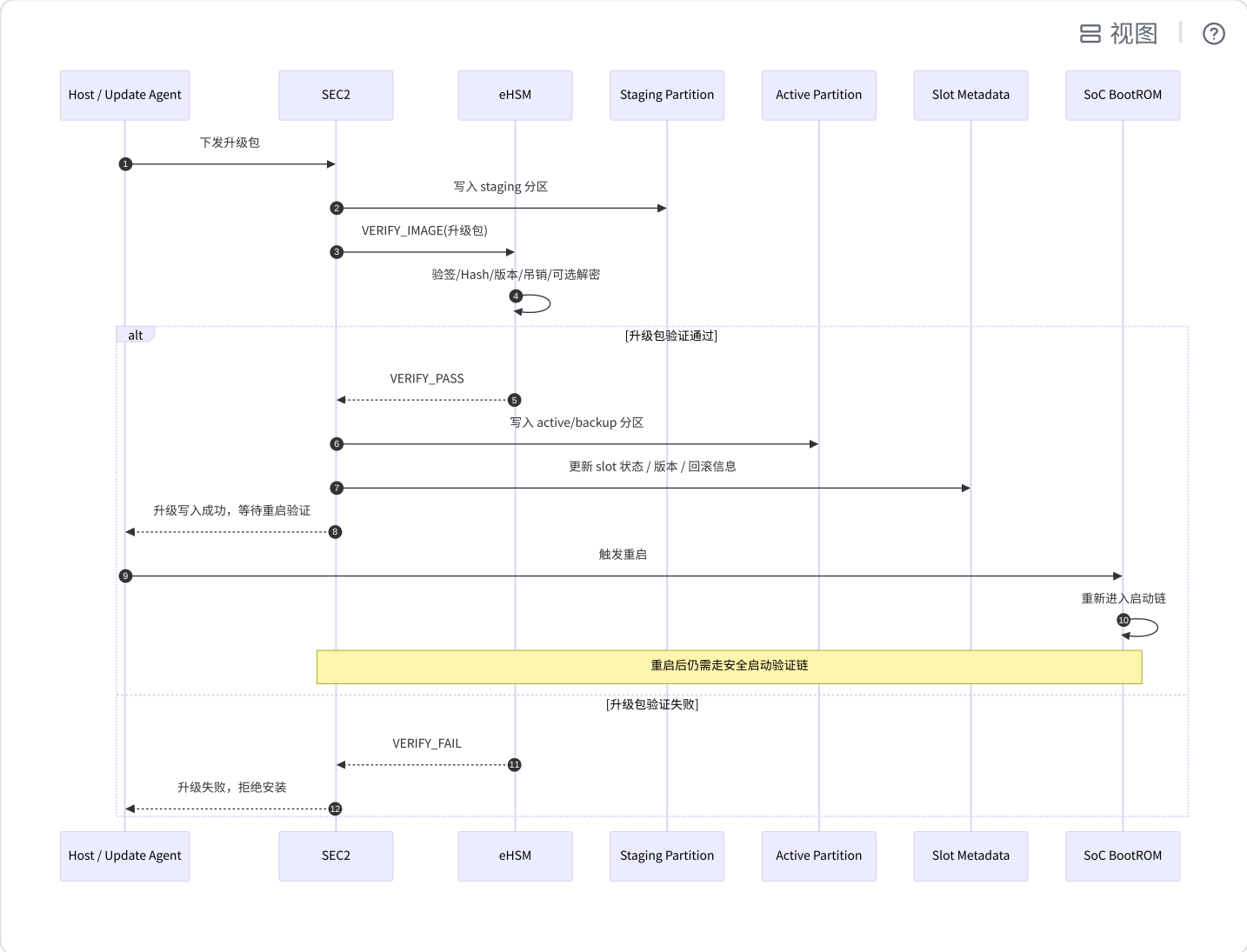
1. 所有升级镜像必须由 SEC2 调用 eHSM 校验
2. Host 只能将升级包写入 缓冲区
3. 启动关键区不得允许 Host 直接写入

4. 需要支持 recovery / backup / rollback
5. 需要支持版本控制与 anti-rollback

10.3 升级流程

1. Host 通过 PLDM 或自定义通道将升级包发送给 SEC2
2. SEC2 将镜像写入 缓冲区
3. SEC2 调用 eHSM：
    - 验签
    - HASH 校验
    - 必要时解密
4. 校验通过后，SEC2 才允许写入 active / backup 分区
5. 更新 slot metadata / counter / version policy
6. 重启后重新进入安全启动链验证
7. 若新镜像失败，则回滚至旧镜像

Host API 中的 `ehsm_verify_image` 和 `ehsm_upgrade_fw_image_*` 可作为这类能力的底层支持。



11. 设备认证与度量方案

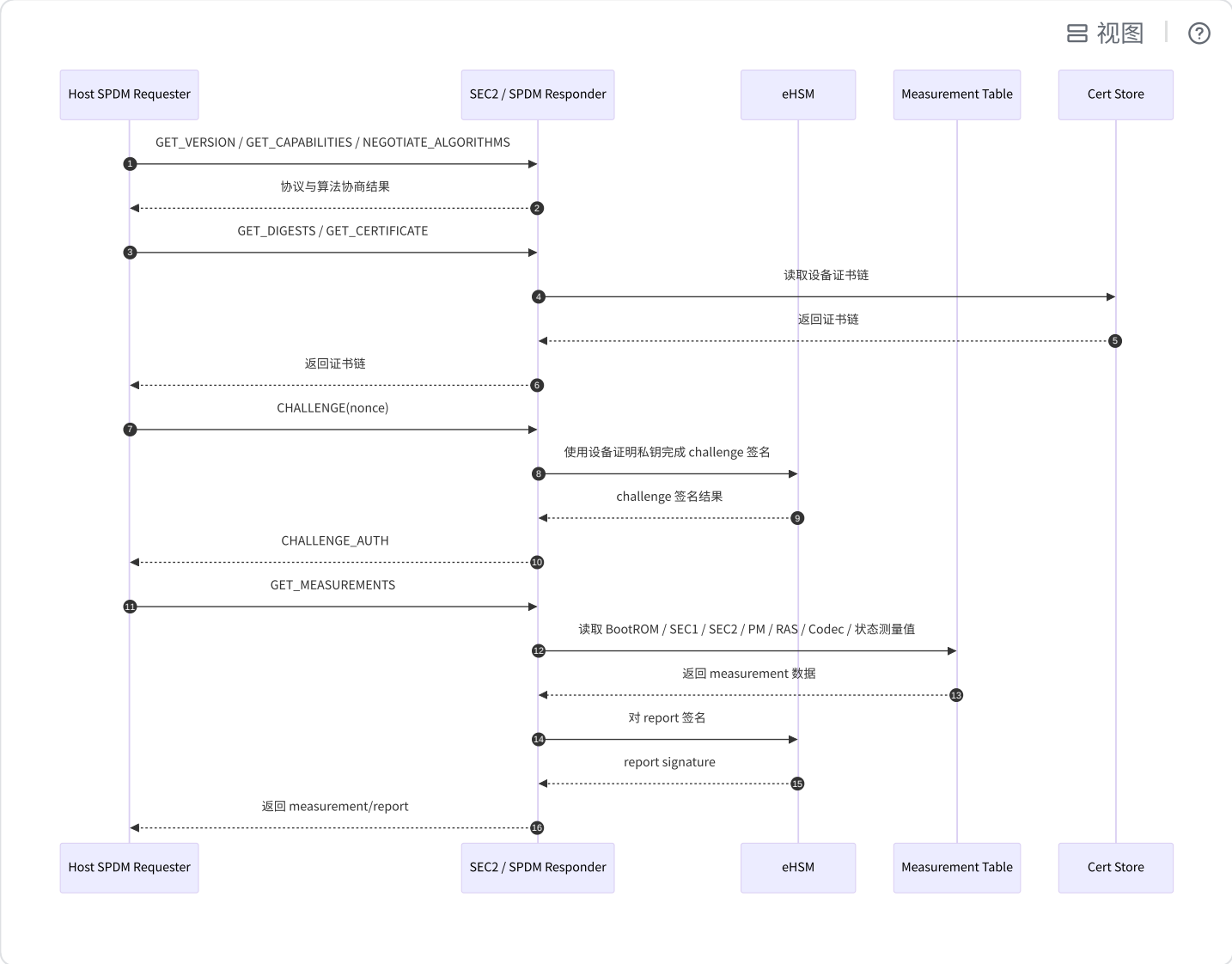
设备远程认证用于向主机侧驱动、管理程序或远端服务器证明：当前 NGU800P 设备的**硬件身份、启动链、关键固件、生命周期状态、调试状态和安全配置**均处于可信状态，且该证明是针对本次挑战实时生成的，而非旧报告重放。

结合当前主流设备认证方案及 NGU800P 的实际架构，本方案采用 **SPDM 协议框架** 作为 Host 与 Device 间的认证基础框架，并结合**国密算法体系（SM2 / SM3 / SM4）**完成设备证书认证、签名验证、会话密钥协商、报告完整性保护及安全会话管理。

11.1 总体架构

NGU800P 设备远程认证涉及三个逻辑实体：**Requester、Responder、Verifier**。

- Host 为 Requester
- SEC2 为 Responder
- Verifier 为 Host 本地策略模块或远端服务
- eHSM 仅提供密钥、签名、摘要、随机数等底层能力



11.1.1 Requester：Host 驱动 / Host 安全代理

Requester 运行于主机侧，由 PCIe 驱动中的安全认证模块承担，主要职责包括:

- 枚举并发现 NGU800P 设备
- 发起 SPDM 协议交互
- 获取证书链、挑战设备、建立安全会话
- 请求测量值和证明报告
- 将报告上传给本地策略模块或远端验证服务
- 根据验证结果决定是否允许驱动继续初始化、分配资源、开放业务队列

11.1.2 Responder：freertos(sec)

Responder 运行于设备安全域中的 freertos(sec)，它是本方案中**唯一允许接触设备证明私钥和根派生密钥的执行体**。其职责包括：

- 作为 SPDM Responder
- 管理静态设备证明证书链

- 管理设备证明私钥
- 汇总 BootROM 和 SEC 阶段产生的 measurement_table
- 组装证明报告
- 使用设备证明私钥对报告签名
- 对外提供证书链、Challenge 响应和测量值

11.1.3 Verifier：本地策略模块或远端验证服务

Verifier 用于最终判断设备是否可信。它既可以是 Host 本地的策略模块，也可以是云平台或服务器侧的远端验证服务。其职责包括：

- 验证设备证书链
- 验证签名与 nonce
- 检查证书有效期、吊销状态、SVN/版本阈值
- 比对测量值与官方reference(**reference 与最小版本门限由厂商发布侧维护，并通过 Host 本地策略数据库或远端验证服务下发)**
- 输出设备信任结论（可信 / 不可信 ）

11.2 密钥与证书体系

11.2.1 设计原则

将“证书链”和“度量链”分开处理：

- **证书链用于证明“设备是谁”**
- **度量链用于证明“设备当前运行状态是什么”**

因此，本方案不要求启动过程中每一级固件生成一张新的 Alias 证书，而是采用：

- **静态设备身份证书链**
- **固定 measurement_table**
- **运行时统一签名报告**

11.2.1 设备唯一根

设备在制造阶段获得设备唯一根材料（HUK / UDS）。其基本要求如下：

- 每设备唯一；
- 不可导出；
- 仅安全域可访问；
- 生命周期绑定；

- 不直接对外签名普通业务数据。

设备唯一根的用途主要包括：

- 设备身份初始化；
- 本地敏感数据 sealing；
- 调试授权绑定；
- 设备安全属性保护。

11.2.2 设备证明私钥

设备证明私钥用于：

- SPDM Challenge 响应签名；
- 证明报告签名。
- 制造阶段生成并安全导入
  - 制造阶段由设备侧生成 Device Attestation KeyPair；
  - 导出公钥或 CSR；
  - 由厂商 CA/HSM 签发设备证明证书，存于设备安全域中；
  - 私钥存于设备安全域中（flash或者NVM），不可导出。

11.2.3 证书链组成

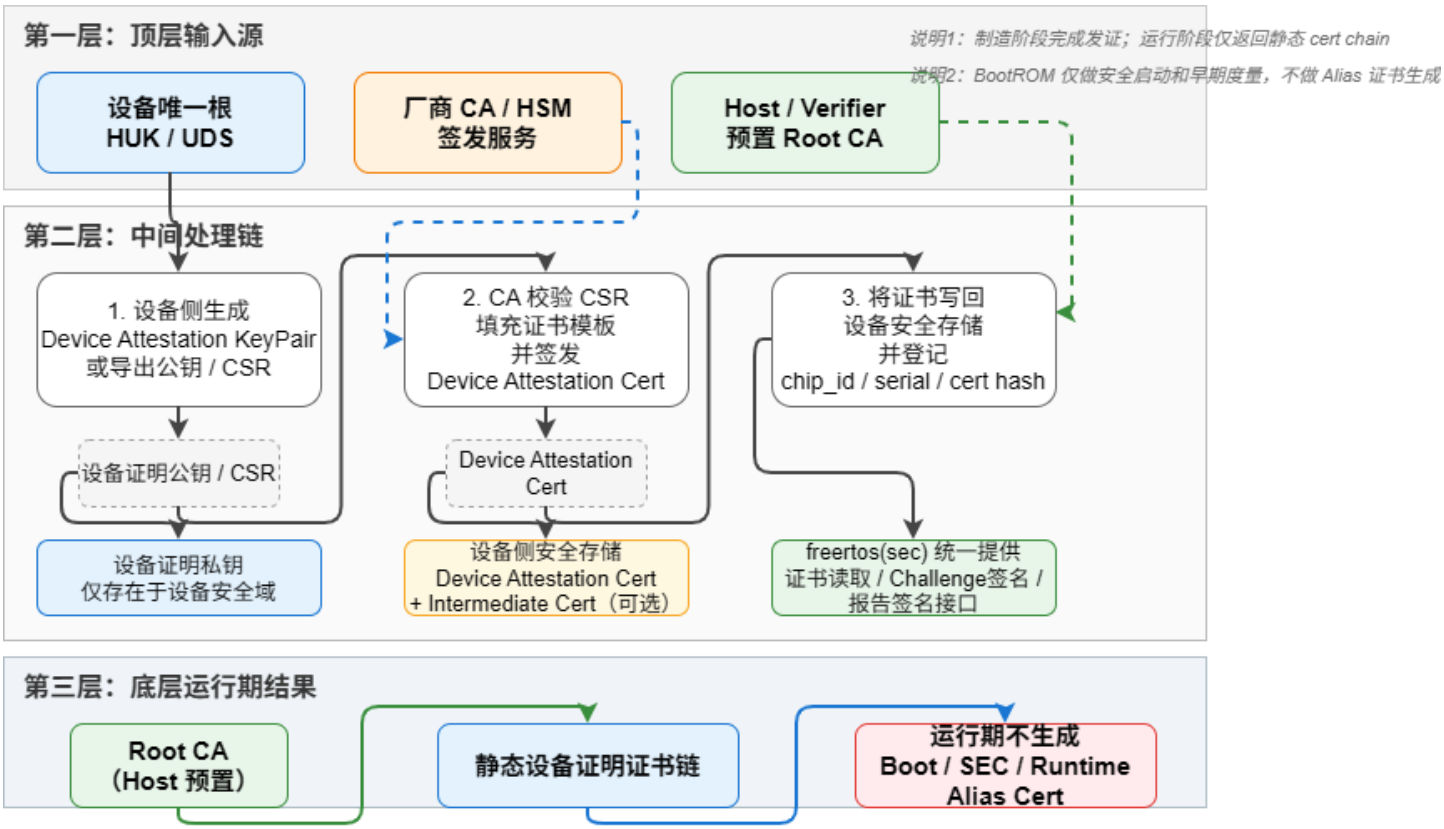
设备认证证书链由以下部分组成：

1. **Root CA Cert**
   - 由我司离线维护；
   - Root CA 私钥不进入设备；
   - 作为整条证书链的最终信任锚；
   - 由 Host / Verifier 预置。
2. **Manufacturer CA / Intermediate CA（可选）**
   - 用于设备证明证书签发；
   - 是否存在取决于内部 PKI 结构；
   - 若存在，则由 Root CA 签发。
3. **Device Attestation Cert**
   - 对应设备证明公钥；
   - 在制造阶段由 Manufacturer CA 或 Intermediate CA 签发；
   - 在设备生命周期内保持稳定；

- 运行时由设备侧返回给 Host。

11.3 证书链生成流程

本节说明证书链在各启动阶段的生成与收敛过程。



11.3.1 制造阶段流程

11.3.1.1 制造阶段目标

制造阶段需要完成以下工作：

- 初始化设备唯一根材料， 存于efuse中；
- 配置安全启动信任锚；
- 生成设备证明公钥 / CSR；
- 签发并回写静态设备证明证书，存于安全flash中；
- 锁定量产态生命周期与调试策略。

11.3.1.2 详细流程

Step 1：设备初始化与根材料准备

- 芯片进入 provisioning / manufacturing 状态；
- 生成或注入设备唯一根材料（UDS）；
- 将其固化到 eFuse 或等效安全存储；

- 配置为不可导出、仅安全域可访问。

Step 2：安全启动信任锚配置

制造阶段同时写入启动验证所需的信任材料，包括但不限于：

- 固件签名公钥 hash 槽位；
- FW_KEK slot / key_id / revoke 配置；
- lifecycle 初始状态；
- anti-rollback 初始值。

Step 3：生成设备证明公钥 / CSR

设备进入安全 provisioning 流程后：

- 生成设备证明 KeyPair，或从设备唯一根派生设备证明公钥；
- 导出设备证明公钥或 CSR；
- 将 chip_id / device_uuid / 产品标识等与 CSR 绑定。

Step 4：厂商 HSM 签发 Device Attestation Cert

制造系统将 CSR 提交给厂商 HSM 平台，由其完成：

- CSR 合法性检查；
- 证书模板匹配；
- 证书用途和有效期填充；
- CA 签名。

输出结果为：

- Device Attestation Cert；
- 可选的 Intermediate CA Cert。

Step 5：证书回写与登记

将签发后的设备证明证书写回设备安全存储，或保存到设备安全分区，并在制造数据库中登记：

- chip_id；
- cert serial；
- cert hash；
- manufacture batch；
- provision profile；
- lifecycle state。

Step 6：量产锁定

在完成安全材料和证书写入后，执行：

- 禁用秘密读回；
- 收紧或关闭 debug；
- 切换 lifecycle 到 PROD；
- 执行一次启动与证书读取冒烟测试。

11.3.1.3 制造阶段输出物

制造阶段最终输出如下安全资产：

- HUK / UDS（设备内部，不导出）；
- 启动验证信任锚（公钥 hash / FW_KEK slot）；
- Device Attestation Cert；
- 可选的 Intermediate CA Cert；
- device_uuid / chip_id 与证书绑定关系；
- 制造审计记录。

11.3.2 BootROM 的职责与工作量

11.3.2.1 设计原则

BootROM 的职责限定为：

- 验证最早级固件；
- 记录最早级度量；
- 将测量结果写入固定表；
- 跳转执行 freertos(sec)。

11.3.2.2 BootROM 具体工作内容

BootROM 阶段完成以下动作：

1. 解析安全核固件包；
2. 验证 SEC 固件签名；
3. 完成 SEC 固件解包 / 解密；
4. 计算 SEC 固件 hash；
5. 记录以下信息到固定 measurement_table 或共享 SRAM：
    - ROM version；
    - SEC fw hash；

- SEC fw version；
- rollback counter。

6. 检查并更新启动状态标志；
7. 跳转执行 freertos(sec)。

11.3.3.1 sec2 作为认证主体

在推荐落地版中，freertos(sec) 是设备认证的统一执行体，负责：

- 验证后续微核固件；
- 更新 measurement_table；
- 管理静态 Device Attestation Cert；
- 管理设备证明私钥；
- 响应 SPDM 证书、Challenge、Measurements 请求；
- 组装并签名完整证明报告。

11.3.3.2 后续固件度量

sec2 在验证各级固件时，逐级记录：

11.3.3.3 平台状态度量

除各级固件外，sec2 还应统一维护状态项：

11.4 认证对象与度量内容

11.4.1 度量原则

设备远程认证的价值不在于返回一个签名本身，而在于签名所覆盖的**实际度量对象**。NGU800P 的认证报告覆盖从安全启动到运行态中最关键的可信对象，确保验证方能够基于这些对象判断当前设备是否符合预期版本和策略。

11.4.2 度量值

freertos(sec) 维护固定的 measurement_table，包含如下内容：

1. **BootROM / immutable identity**
   - ROM version

   bootrom阶段将版本信息存入指定sram处。

2. **安全管理固件 freertos(sec)**
   - sec fw hash（SM3）

- sec fw version
- rollback counter

bootrom加载sec1 固件时将信息及测量结果存入指定sram处

3. **其他微核固件集合**
   - 微核固件 hash 列表
   - 对应 version
   - rollback counter

   sec2校验固件时，将信息及测量结果存入
4. **安全状态**
   - lifecycle state
   - debug state
   - secure boot enable
   - anti-rollback enable

   freertos(sec) 统一管理状态信息
5. **平台标识**
   - chip_id

度量值由启动链各阶段在验证时产生，并最终由 sec2 统一接管、汇总、维护和对外输出。

11.5 报告内容与格式

设备输出的证明报告为一份结构化数据对象。报告包含以下几个部分。

- report header
- measurement集合
- cert chain
- signature block

11.5.1 报告头

报告头用于描述报告整体属性，定义如下：

代码块

```
struct ngu_attest_report_header{
    uint32_t magic;
    uint32_t header_len;
    uint32_t total_len;

```

```
    uint8_t  report_uuid[16];
    uint8_t  device_uuid[16];
    uint8_t  requester_nonce[32];

    uint32_t lifecycle_state;
    uint32_t debug_state;
    uint32_t session_id;

    uint32_t hash_algo;
    uint32_t sig_algo;
    uint32_t cert_format;

    uint32_t block_count;
    uint32_t signed_region_offset;
    uint32_t signed_region_len;
};
```

- `magic`
  - 报告魔数，用于识别认证报告格式。建议固定为：
- `header_len`
  - Report Header 自身长度，便于后续扩展 header 字段。
- `total_len`
  - 整个认证报告的总长度，包括 Header、Directory、所有 Block 和 Signature Block。
- `report_uuid`
  - 本次报告实例 ID，用于唯一标识一份报告。每次生成报告时随机产生。
- `device_uuid`
  - 设备唯一标识，绑定芯片或板卡实例。
- `requester_nonce`
  - 由 Host 发起认证请求时提供的随机数。
  - 该字段必须原样写入报告，并纳入签名覆盖范围。
- `lifecycle_state`
  - 设备当前生命周期状态，与 eFuse状态编码保持一致，如DEV/PROD/RMA
- `debug_state`
  - 设备当前 Debug 状态，建议编码反映：
- `session_id`
  - 报告绑定在 SPDM 安全会话中，填写会话 ID；若未建立正式会话，可置0。

- `block_count`
  - 后续 block 的数量。
- `signed_region_offset` / `signed_region_len`
  - 用于显式定义签名覆盖区域，便于 Host 侧统一计算摘要。
  - `signed_region_offset = 0`
  - `signed_region_len = Signature Block 起始前的全部长度`

11.5.2 测量块

Measurement Block 是认证报告的核心，用于承载安全启动和运行态的可信对象。

1）Measurement Block Header

代码块

```
typedef struct {
    uint16_t block_type;
    uint16_t block_version;
    uint32_t block_len;

    uint32_t flags;
    uint32_t reserved0;
} ngu_measurement_block_header;
```

`block_type`：固定为 measurement block

`block_version`：当前版本

`block_len`：整个 measurement block 总长度

`flags`：可选，标识这个 block 的整体属性

其他预留

2）fw info

代码块

```
typedef struct {
    uint32_t fw_type;
    uint8_t  hash[32];              // SM3
    uint32_t fw_version;
    uint32_t rollback_counter;
    uint32_t reserved0;
} ngu_meas_fw_info;
```

3) 设备状态

代码块

```
typedef struct {
    uint32_t lifecycle_state;
    uint32_t debug_state;
    uint32_t secure_boot_enable;
    uint32_t anti_rollback_enable;
    uint8_t chip_id[16];
    uint32_t reserved0;
} ngu_meas_state;
```

4）Measurement Block

代码块

```
typedef struct {
    ngu_measurement_block_header hdr;

    ngu_meas_fw_info  bootrom_info;
    ngu_meas_fw_info  sec_fw_info;
    ngu_meas_fw_info  aux_fw_info[MAX_AUX_FW];
    ngu_meas_state    plat_state;
} ngu_measurement_block;
```

11.5.3 证书链块

1) Cert Chain

证书链的格式采用SPDM 风格的整链 blob，DMTF 的 SPDM 架构文档明确提到：返回的 certificate chain 包含 root certificate 的 hash；还建议实现者把整条证书链（包括 root cert） 存在 slot 里，以便按当前协商的hash 算法生成这个 hash。

每张固件的证书采用 DER 编码 **X.509 v3证书。**

代码块

```
typedef struct __attribute__((packed)) {
    uint16_t length;
    uint16_t reserved;
    uint8_t  root_hash[H];
    uint8_t  cert_chain[];
} spdm_cert_chain_blob_t;
```

- `length`
  - 这是整条 chain blob 的总长度
- `root_hash`
  - Root certificate 的 hash
- `cert_chain[]`
  - 证书链中的每张证书都应为 ASN.1 DER-encoded X.509 v3
  - 就是 [Device Attestation Cert] [Intermediate CA Cert (optional)]

2）Cert Chain Block

代码块

```
typedef struct __attribute__((packed)) {
    uint16_t block_type;
    uint16_t block_version;
    uint32_t block_len;

    uint32_t cert_chain_format;   // 例如固定为 SPDM_CERT_CHAIN_BLOB
    uint32_t cert_chain_len;      // 后续 blob 长度
    uint32_t reserved0;

    uint8_t  cert_chain_blob[];   // 这里直接放完整 SPDM chain blob
} ngu_cert_chain_block;
```

11.5.4 签名 Block

Signature Block 用于描述签名算法、签名对象和签名值，是认证报告可信性的最终保证

1）signature_block_header

代码块

```
typedef struct {
    uint16_t block_type;
    uint16_t block_version;
    uint32_t block_len;

    uint32_t sig_len;
    uint32_t signer_id;
} ngu_signature_block_header;
```

2）signature_block

代码块

```
typedef struct {
    ngu_signature_block_header header;
    uint8_t signature[sig_len];
} ngu_signature_block;
```

11.6 SPDM 认证流程

11.6.1 总体说明

NGU800P 的远程认证采用 SPDM 的标准交互阶段组织流程。Host 侧为 Requester，设备安全域 freertos(sec) 为 Responder，部署spdm的软件栈，双方先完成版本/能力/算法协商，再完成证书获取与挑战认证，随后建立安全会话，并在会话中获取测量值和认证报告。

11.6.2 详细流程

1）设备安全启动与度量

设备上电后，BootROM 接收并验证 c908 安全管理固件，验签和解密通过后跳转执行 sec2。随后 sec2 继续接收并验证其他微核固件，并在每一步验证通过后记录对应度量值，最终进入安全监管状态。该流程已在文档第 7 章定义，本节不再重复，只在远程认证时导出其度量结果。

2）版本、能力和算法协商

Host 驱动在 PCIe 枚举完成、基础寄存器可访问后，通过 SPDM 向设备发起：

- `GET_VERSION`
- `GET_CAPABILITIES`
- `NEGOTIATE_ALGORITHMS`

在本项目中，能力协商至少应覆盖：

- 证书能力
- Challenge 能力
- Measurements 能力
- 最大消息长度
- 分片能力（如证书链较长）

算法协商应固定到国密族：

- Base Hash：SM3
- Base Asym：SM2-Sign

需要注意的是，虽然 `libspdm` 的算法矩阵列出了这些算法，但其官方 OpenSSL wrapper 对 `SM2-KeyExchange` 仍标注为不支持，实现上仍需自行补齐 crypto 封装。

3）获取设备证书链

Host 发起：

- `GET_DIGESTS`
- `GET_CERTIFICATE`

设备返回设备证明证书链及其摘要。Host 使用预置的我司 Root CA 对证书链进行校验，验证链条、有效期、用途字段、序列号和吊销状态是否合法。只有在证书链可信的前提下，后续 Challenge 和报告签名才有意义。

4）设备挑战认证

Host 发起 `CHALLENGE`，并在请求中携带随机 `nonce`。设备使用设备证明私钥对规定的 challenge 上下文签名，返回 `CHALLENGE_AUTH`。Host 校验签名后，可确认当前设备确实持有叶子证书对应的私钥，完成“设备身份真实性”认证。

5）建立安全会话

在 Challenge 成功后继续执行：

- `KEY_EXCHANGE`
- `FINISH`

从协议设计上，采用：

- transcript hash：SM3
- key schedule / KDF：SM3-KDF
- 会话流量保护：SM4_GCM

6）请求测量值和认证报告

在通过 Challenge 后，Host 发起 `GET_MEASUREMENTS`，请求设备返回测量值和完整认证报告。设备的freertos(sec) 从安全域 `measurement_table` 中读取各测量内容，补充 lifecycle/debug 等当前状态，生成结构化报告，并使用设备证明私钥完成 SM2 签名后返回。

7）验证与策略判定

Host 或远端验证服务收到报告后，依次完成：

- 验证证书链；
- 验证报告签名；
- 检查 `nonce` 是否与本次请求一致；
- 检查 lifecycle 是否允许上线；
- 检查 debug 状态是否关闭或处于允许模式；
- 检查各测量值是否与我司 measurements 匹配；

- 检查版本是否不低于最小允许门限。

若校验全部通过，则设备被判定为可信，驱动可继续初始化；若失败，则应禁止创建业务队列、禁止进入集群或进入受限模式，并输出告警日志。

11.7 Device && HOST 侧软件模块划分

11.7.1 Host 侧模块

Host 侧建议拆分为以下四个模块：

- `gpu_spdm_transport`
  - 负责 Host 与 Device 之间的底层消息收发
  - 对接 PCIe BAR / doorbell / mailbox / shared buffer
  - 处理请求投递、响应回读、超时、分片重组
- `gpu_spdm_requester`
  - 负责 SPDM Requester 状态机
  - 发起版本协商、能力协商、算法协商
  - 发起证书获取、Challenge、Measurements、Session 建立等流程
- `gpu_attest_verify`
  - 负责证书链校验
  - 负责报告签名校验
  - 负责 nonce 校验、设备状态校验、测量值比对
  - 输出认证结论与失败原因
- `gpu_policy_agent`
  - 负责把认证结果映射为驱动策略
  - 决定是否继续初始化
  - 决定是否允许创建设备队列、开启多租户、进入集群
  - 可选对接远端验证服务器

11.7.2 Device 侧模块

Device 侧由 freertos(sec) 承担认证主体，拆分为以下模块：

- `spdm_transport`
  - 负责解析 Host 投递的 transport 包
  - 管理 RX/TX buffer

    - 管理 doorbell/中断/轮询机制
- `spdm_responder`
    - 负责 SPDM Responder 状态机
    - 根据请求类型分发到证书、挑战、测量、会话等处理函数
    - 维护连接上下文与会话上下文
- `attest_store`
    - 管理设备证明证书链
    - 管理设备证明私钥装载、解包、清理
    - 提供证书摘要、证书链读取接口
- `measurement_mgr`
    - 维护 `measurement_table`
    - 提供启动阶段度量写入接口
    - 提供运行态状态更新接口
    - 提供证明报告组装接口
- `crypto_hal`
    - 封装国密算法接口
    - 提供 SM2 签名/验签、SM3 摘要、SM4-GCM、KDF 等能力
    - 屏蔽底层软实现或硬件加密引擎差异
- `policy_mgr`
    - 根据 lifecycle/debug/svn/撤销位图控制认证输出和行为
    - 决定某些敏感字段是否允许上报。

12. 调试鉴权与生命周期

12.1 调试鉴权原则

调试接口默认必须关闭，仅在允许生命周期中，通过 challenge / credential / signature 流程完成鉴权后才能打开。

12.2 生命周期映射

eHSM 原生定义了 TEST、DEVELOP、MANUFACTURE、USER、DEBUG、DESTROY 等生命周期。

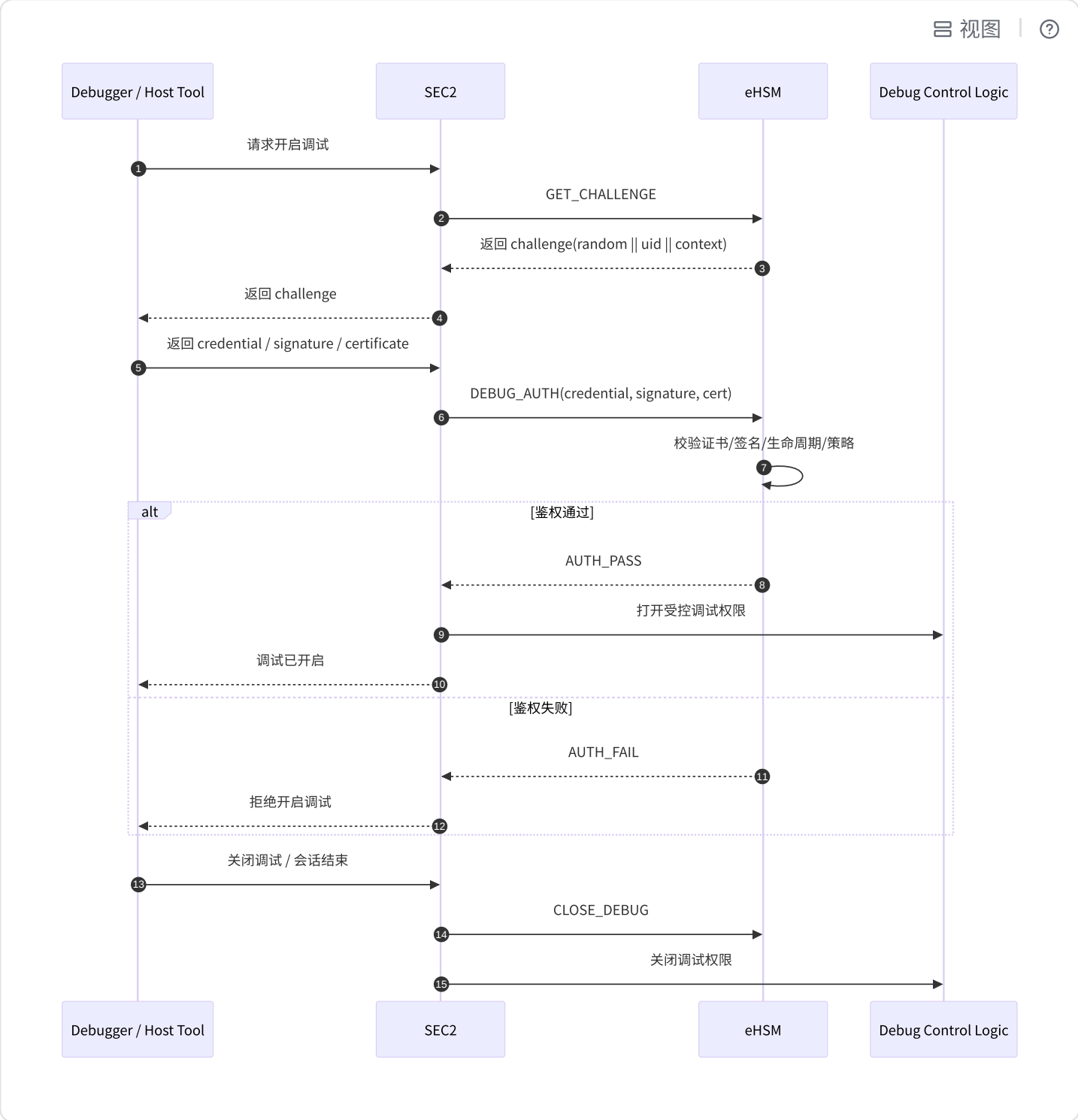
eHSM 生命周期	SoC 建议名称	说明
TEST	产测态	芯片筛选、测试
DEVE	开发态	样片与固件开发
MANU	板级开发态	客户开发调试
USER	量产态 / PROD	正式运行态
DEBG	RMA / 返修态	失效分析与受控调试
DEST	销毁态	不可恢复使用

12.3 调试鉴权流程

eHSM 文档明确支持 challenge-response 调试鉴权，支持 Mailbox 和 UART 通道；Host API 中也有 `ehsm_get_challenge`、`ehsm_debug_auth`、`ehsm_close_debug` 等能力。

本方案建议：

1. Host 调试工具向 SEC2 发起调试授权请求
2. SEC2 调用 eHSM 获取 challenge
3. Host 返回凭证/签名
4. SEC2 调用 eHSM 完成校验
5. 校验成功后由 SEC2 打开受限调试通路
6. 会话结束后自动关闭



13. 制造注入与信任材料

13.1 制造阶段需要完成的内容

制造阶段建议完成：

1. 注入或生成设备根材料（UDS/HUK）
2. 写入 signer pubkey hash 槽位
3. 写入 revoke bitmap
4. 写入 anti-rollback 初始值

5. 写入 debug root / debug CA hash
6. 生成设备证明公钥 / CSR
7. 由厂商 HSM / CA 签发 Device Attestation Cert
8. 回写证书链与绑定信息
9. 切换生命周期到量产前完成冒烟验证

13.2 密钥分类

建议系统密钥分为：

1. 启动信任锚类
   - signer pubkey hash slot
   - revoke bitmap
   - key policy
   - FW_KEK / image protect key
   - anti-rollback counter
2. 设备身份类
   - UDS/HUK
   - Device Attestation Private Key
   - Device Attestation Cert
3. 调试类
   - debug root hash / debug CA hash
   - challenge / debug auth 相关密钥

eHSM 文档明确支持密钥分级保护、片内生成、片外加密导入以及按 Key ID 调用。

14. 访问控制与隔离要求

14.1 总体目标

本章节定义 SoC 在运行态及启动态对各类访问主体（Host、SoC CPU、DMA Master、微核等）的访问控制与隔离要求，以满足以下安全目标：

1. 防止 Host 通过 PCIe MMIO 或 DMA 非授权访问设备内部敏感资源
2. 防止普通软件绕过安全启动链执行未验证代码
3. 防止关键安全状态（secure boot、debug、lifecycle 等）被篡改
4. 防止关键存储区域（固件、证书、策略、metadata）被非法修改

5. 防止高危硬件控制能力（电压、时钟、熔丝、调试旁路等）暴露给非安全主体

本章节要求的访问控制能力应需优先**通过 RTL 硬件机制实现**。

14.2 主体分类与控制原则

14.2.1 访问主体分类

系统中所有访问请求必须能够在硬件层区分其来源，至少包括以下类型：

主体类型	描述
PCIe MMIO Master	Host 通过 BAR 访问寄存器
PCIe DMA Master	Host 发起的 DMA 访问
SEC2 Master	安全控制核（最高权限）
普通 SoC CPU	非安全软件执行环境
内部 DMA Master	视频/Codec/Copy Engine 等
微核 Master	PM / RAS / Codec 等

14.2.2 控制原则

本章节通过对 PCIe MMIO、DMA、内部 master、关键寄存器及存储区域的分级隔离与访问控制，确保：

- **Host 仅具备最小必要访问权限**
- **所有安全关键操作必须通过 SEC2 控制路径执行**
- **未授权访问在硬件层被阻断并可审计**

从而构建完整的 SoC 运行态安全隔离体系。

14.3 PCIe MMIO 访问控制要求

14.3.1 功能性开放范围

系统应允许 Host 通过 PCIe MMIO 完成以下功能：

- 设备状态读取
- Doorbell / Queue / Mailbox 交互
- 固件下载描述符配置
- 普通状态与版本信息查询

14.3.2 禁止访问范围

Host 不得通过 MMIO 直接执行以下操作：

- 微核启动（release）控制
- secure boot 状态修改
- debug enable
- recovery 模式选择
- flash 安全写控制
- 生命周期（lifecycle）修改
- 密钥与策略修改
- 安全旁路或测试寄存器控制

14.4 PCIe DMA 访问控制要求

14.4.1 允许访问范围

Host DMA 仅允许访问：

- 固件缓冲区
- 普通数据缓冲区

14.4.2 禁止访问范围

Host DMA 不得访问：

- SEC1 / SEC2 执行区
- 微核运行区
- recovery 区
- 证书/策略区
- 安全共享缓冲区
- 安全状态寄存器区

14.3 安全寄存器域要求

以下寄存器必须属于安全寄存器域，仅允许 SEC2 写入：

- 微核 release 控制
- secure boot 状态
- debug enable
- recovery 选择
- flash 安全写使能
- 安全错误状态

14.3.1 RTL 要求

1. 上述寄存器必须不对 Host 暴露写权限
2. 普通 CPU 不得写入
3. 必须支持 SEC2 独占写访问
4. 部分寄存器应支持硬件状态机控制（如 secure boot 状态）
5. debug enable 必须支持鉴权后开启与自动关闭

14.4 启动关键区写保护要求

14.4.1 保护范围

以下区域必须受写保护控制：

- SEC1 固件区
- SEC2 固件区
- recovery 区
- 证书与策略区
- slot metadata 区

14.4.2 RTL 要求

1. Flash/NVM 控制器必须支持分区级写保护
2. 正常运行态下上述区域必须只读
3. 仅允许 SEC2 在升级窗口内临时开启写权限
4. 写权限必须自动回收
5. metadata 区必须支持原子更新机制

14.5 高危寄存器隔离要求

14.5.1 控制范围

以下寄存器属于高危控制域：

- 电压调节控制
- 熔丝（eFuse）控制
- 时钟控制（PLL / bypass / gate）
- 安全旁路寄存器
- 调试旁路控制

14.5.2 RTL 要求

1. 不得映射到 Host 可访问地址空间
2. 普通 CPU 不得访问
3. 仅允许安全路径（SEC2 或 eHSM）控制
4. 必须与 lifecycle 状态绑定
5. 对于旁路寄存器：
    - 量产态必须锁定
    - 不得通过软件解除
6. 调试控制必须满足：
    - 默认关闭
    - 必须通过 eHSM 鉴权开启
    - 支持自动关闭

15. 客户需求映射表

客户需求	设计落点
固件支持数字签名	SEC1 首次由 eHSM 验证；SEC2 和后续微核由 SEC2 调用 eHSM 验证
不得使用已泄漏签名私钥继续签发固件	发布私钥位于厂商 HSM；设备侧保存 pubkey hash slot + revoke bitmap + key_id 策略
签名私钥存储在 HSM，确保可用不可见	设备运行私钥、设备证明私钥、调试鉴权私钥存于 eHSM；发布私钥不入设备
支持基于升级可信根的签名升级校验	SEC2 作为升级控制平面，调用 eHSM 完成镜像验证
禁止绕过 RTU 的升级路径	Host 仅可写 缓冲区，关键区写入必须经 SEC2 授权
防止软件直接操作 flash 导致不可恢复	启动关键区写保护 + recovery / backup / rollback 机制
基于硬件可信根完成链式验签	BootROM 启动编排，eHSM 首次验证 SEC1，SEC2 继续验证后续固件
验签公钥写 OTP，量产后受限	signer hash slot / revoke / key policy 写入 OTP/eFuse
支持密钥吊销更新	key_id + 多 slot + revoke bitmap
PCIe MMIO 加载的小核固件必须验签	Host 只能投递镜像，执行门控由 SEC2 控制
支持 SPDM	SPDM 协议栈运行于 SEC2
支持部件度量认证	measurement_table + cert chain + report + signature
PCIe MMIO/DMA 地址空间需可控	总线防火墙 + 地址窗口 + master 权限矩阵
内部管理逻辑不可被用户触达或篡改	安全寄存器域只允许 SEC2 写
防止高危软件改写控制逻辑	电压/安全旁路/熔丝/高危寄存器隔离
禁止绕过内存访问控制接口	DMA/MMIO 统一受防火墙约束
禁止存在绕过系统安全机制的功能	安全服务统一收口到 SEC2 ↔ eHSM
禁止隐藏、不可管理、未文档化接入方式	建立寄存器矩阵、接口清单、量产关闭清单
禁止恶意软件	secure boot + secure update + execution gating
禁止私有算法	统一采用 SM2/SM3/SM4 或标准算法族
工作密钥不能硬编码	工作密钥由 eHSM 生成/派生，软件仅引用 key_id/handle
随机数必须安全	nonce / challenge / CEK / report_uuid 等由 eHSM TRNG 生成
漏洞需要扫描和修复	技术侧配套最低版本门限、升级与吊销能力
漏洞需及时通知与计划	研发流程和版本基线管理